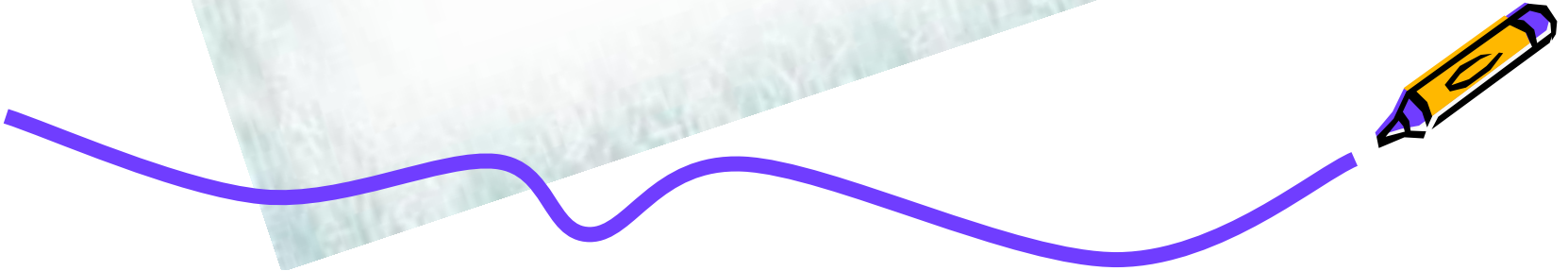




Chapter 10 Update Transactions



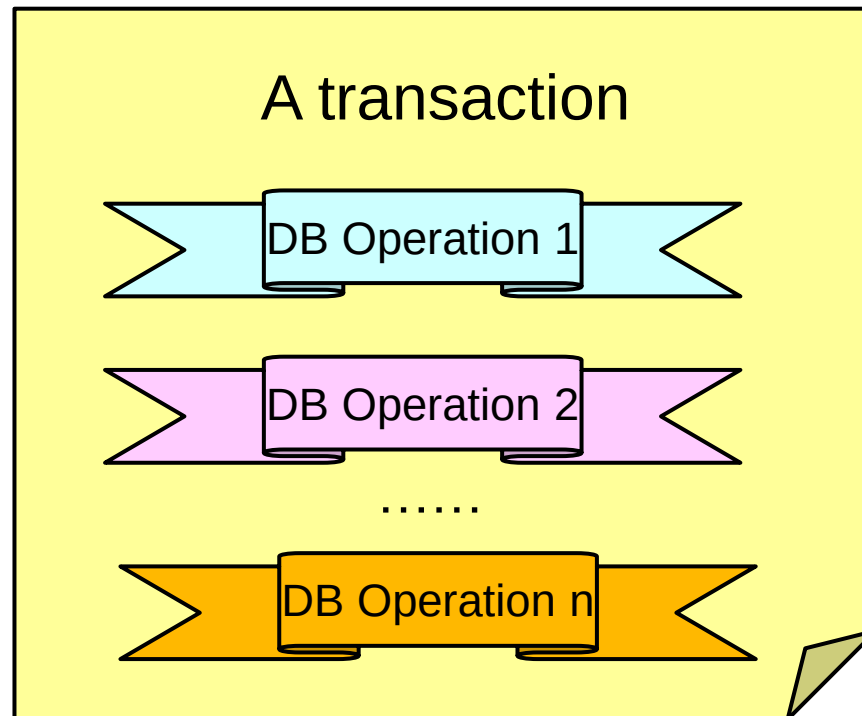
Update Transaction



School of software Yunnan University

DEFINITION 10.1 Transaction

- A *transaction* is a means by which an application programmer can package together a sequence of database operations.

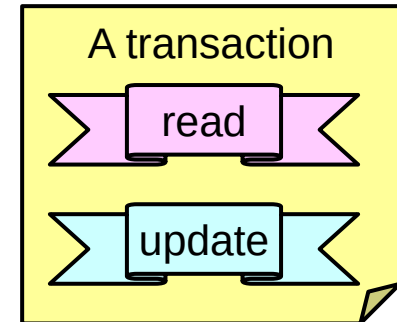


Update Transaction

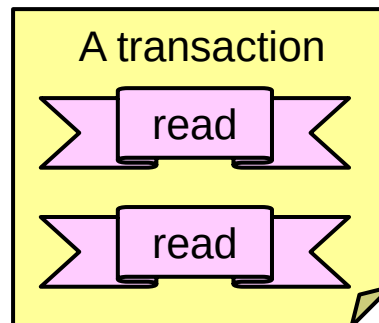


School of software Yunnan University

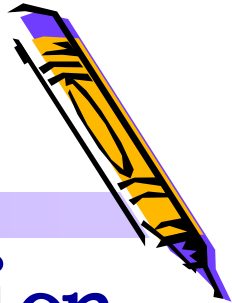
- When the operations making up a transaction consist of both **reads** and **updates**, they represent an attempt by the application programmer to perform **a consistent change of state in the data**;



- when the operations of a transaction consist only of **reads**, they represent an attempt at **a consistent view of the data**.



Update Transaction



School of software Yunnan University

1950s~1970s Transaction

- To write large database applications, early developers faced the following problem:
 - Errors of concurrent execution.
 - Creating an inconsistent result.
 - Uncertainty as to when changes become permanent



- 常见的并发问题

Lost Update (丢失更新)

- 含义：两个事务更新相同的数据，它们均修改并保存数据，这将出现丢失更新的问题。对数据的最后一次成功更新将保存下来，而第一次更新将被覆盖。

丢失更新问题



School of software Yunnan University

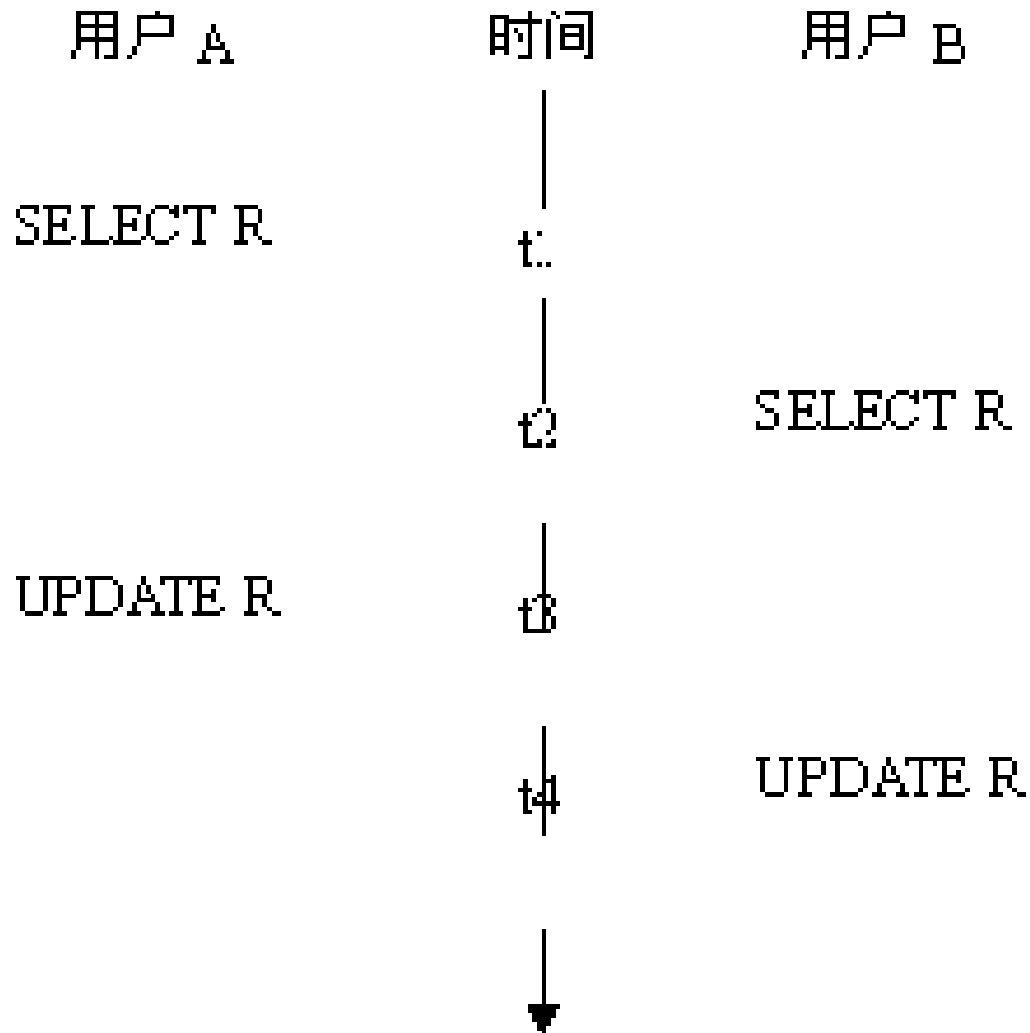
□ 旅客A来到A售票处，要买一张15日昆明到北京的61次直达快速列车的软卧车票，售票员A（下称用户A）在终端A查看剩余票信息；

总的效果：15日61次8车厢5号下铺的软卧票卖出了两次。其原因是，允许了用户B在过时的信息基础上更新数据库，而没有迫使他去看最新的票信息。

□ 旅客A买了一张15日61次8车厢5号下铺的软卧票，用户A更新剩余票信息并将它存入数据库；
□ 这时用户B不知道用户A已经将15日61次8车厢5号下铺的软卧票卖出，使旅客B也买了一张15日61次8车厢5号下铺的软卧票，用户B更新剩余票信息并将它存入数据库（重复了用户A已经做过的更新）。

丢失更新问题

School of software Yunnan University





□ 常见的并发问题

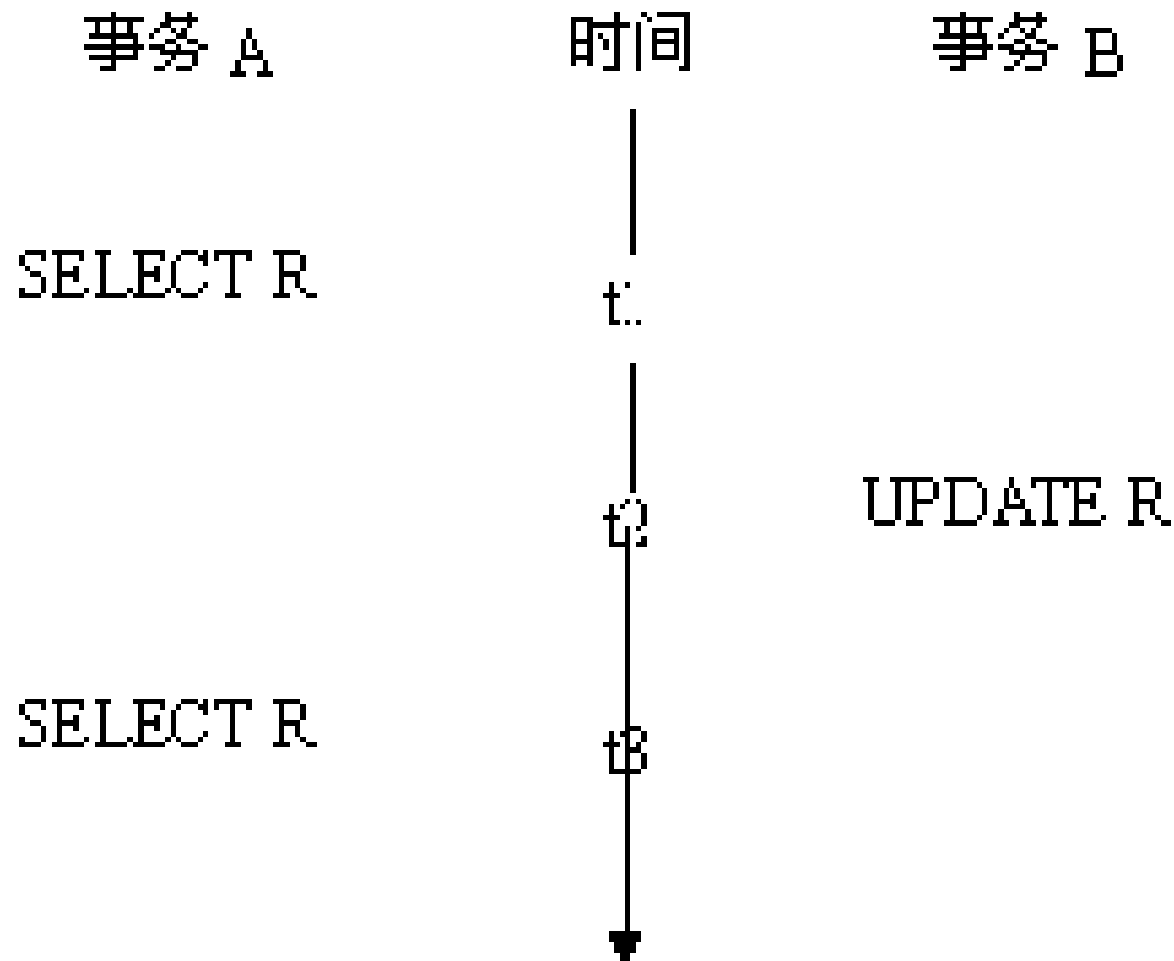
Nonrepeatable read (不可重复读)

- 含义：事务T1读数据后，事务T2执行更新操作，使T1无法再现前一次读取结果。
- 很多应用可能需要校验功能，这时往往需要连续两次或多次读数据进行校验和分析，结果由于其他事务的干扰，使得前后结果不一致，从而产生校验错误。

不可重复读问题



School of software Yunnan University





□ 常见的并发问题

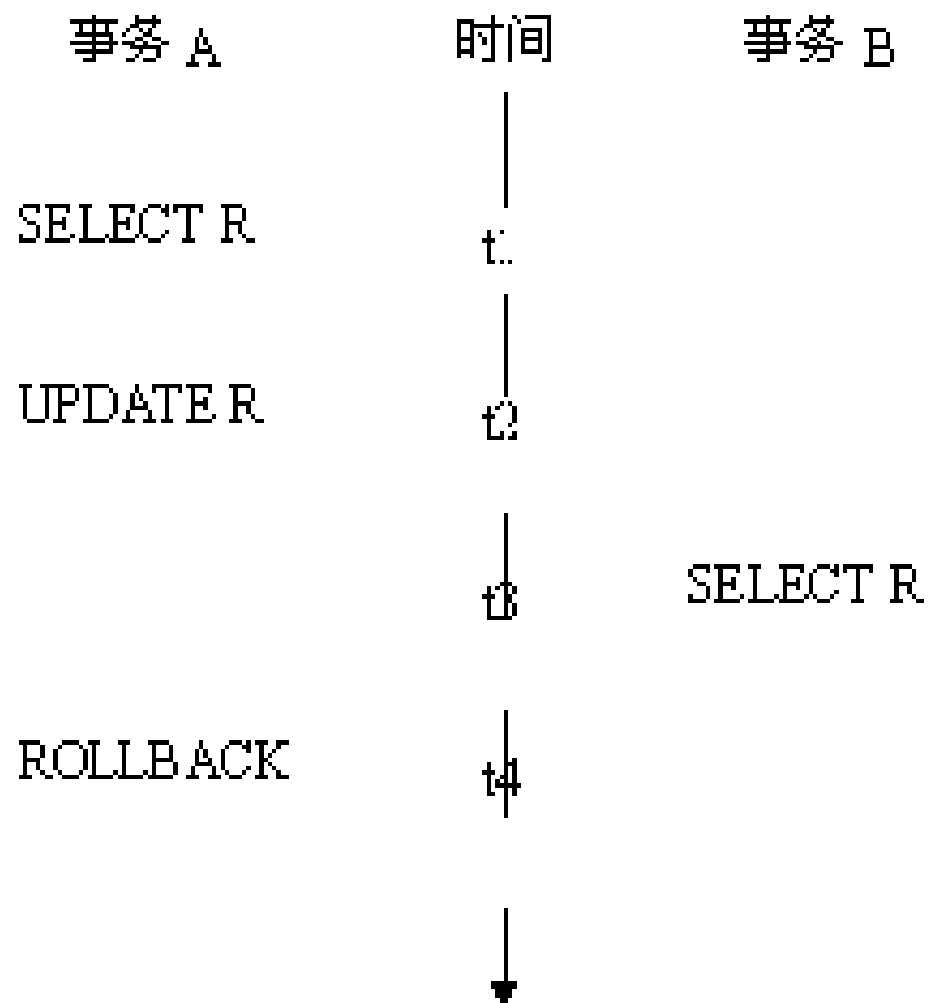
未提交读(Uncommitted read)

- 含义：事务T1修改某一数据，并将其写回磁盘，事务T2读同一数据后，T1由于某种原因被撤消，这时T1已修改过的数据恢复原值，T2读到的数据就与数据库中的数据不一致，则T2读到的数据就为“脏”读。

读“脏数据”问题



School of software Yunnan University





□ 常见的并发问题

phantom read (幻象读)

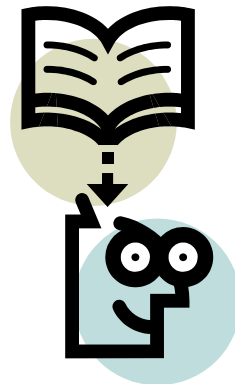
- 事务T1按一定条件从数据库中读取了某些数据后，事务T2删除了其中部分记录，当T1按相同条件再读取数据时，发现某些记录消失了。
- 或者事务T2插入了一些记录，当T1再查询时，发现多了一些记录。

Update Transaction



School of software Yunnan University

- The database system makes the following four transactional guarantees, known as the *ACID guarantees*, or *ACID properties*.
- The term *ACID* is an acronym for four properties : **Atomicity**, **Isolation**, **Durability**, and **Consistency**.



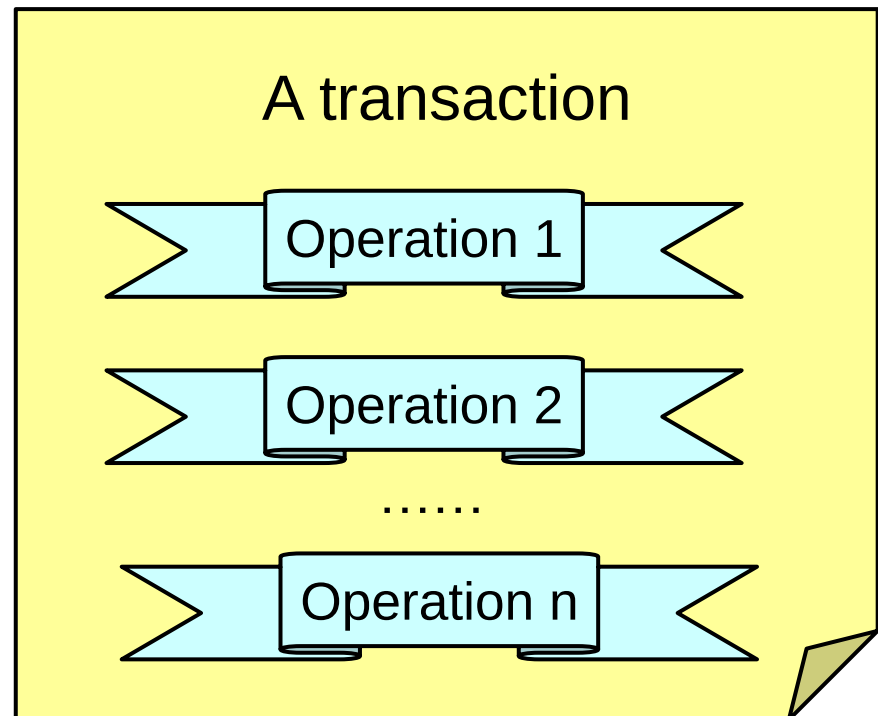
Update Transaction



School of software Yunnan University

■ Atomicity

- A set of record updates that are part of a transaction is indivisible.
- Thus either all updates of a transaction occur in the database, or none of them occurs.

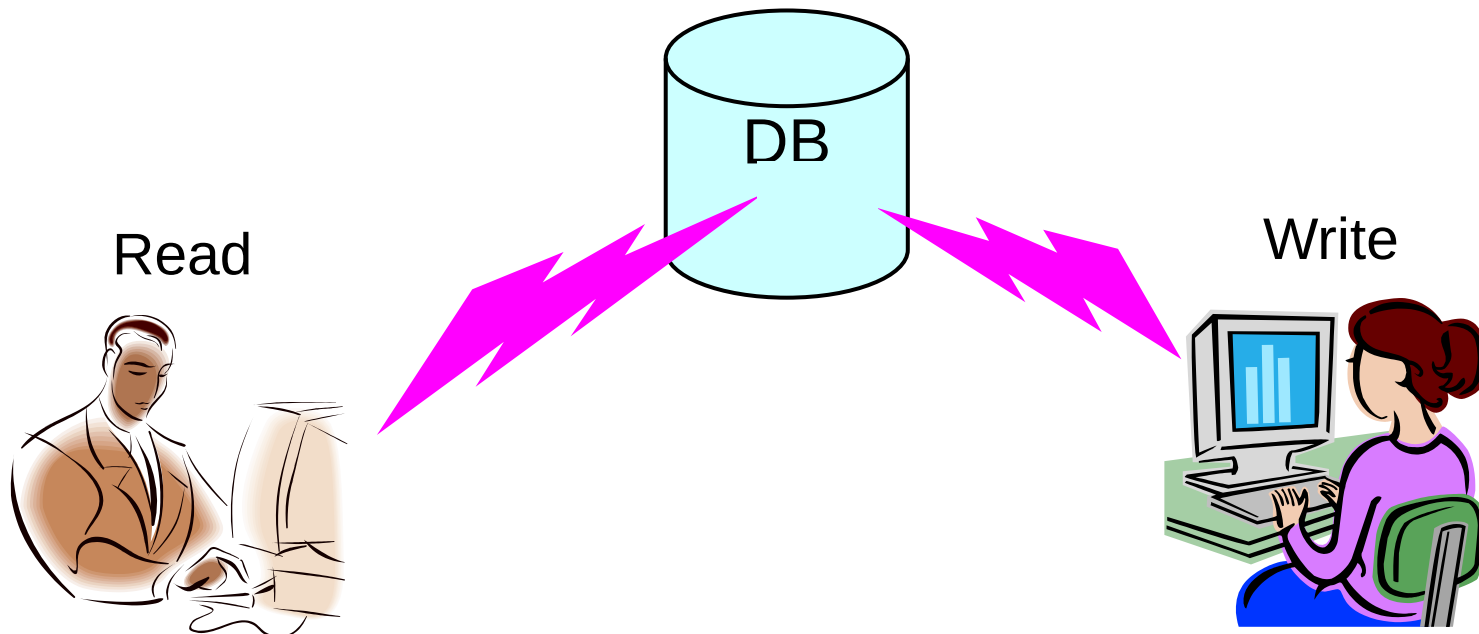


10.1 Transaction Histories



School of software Yunnan University

- When two or more users perform interleaved operations of read and write on the database,
- we need **isolation** of transactions.



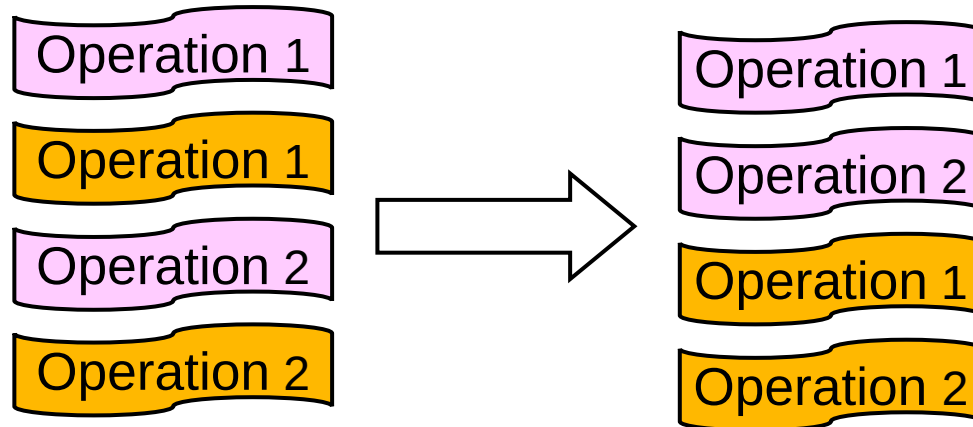
Update Transaction



School of software Yunnan University

■ Isolation

- It also called is serializability, meaning that any schedule of interleaved operations permitted by the system is equivalent to some serial schedule.



lock

Update Transaction

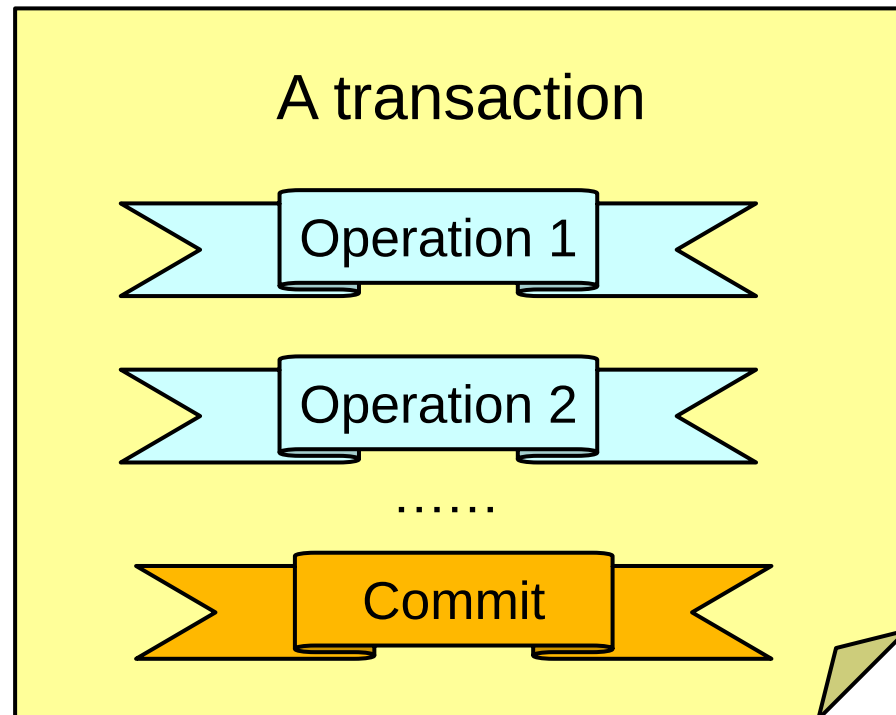


School of software Yunnan University

■ Durability

- when the system returns to the program logic after a *Commit Work* statement, the transaction is guaranteed to be durable.

log



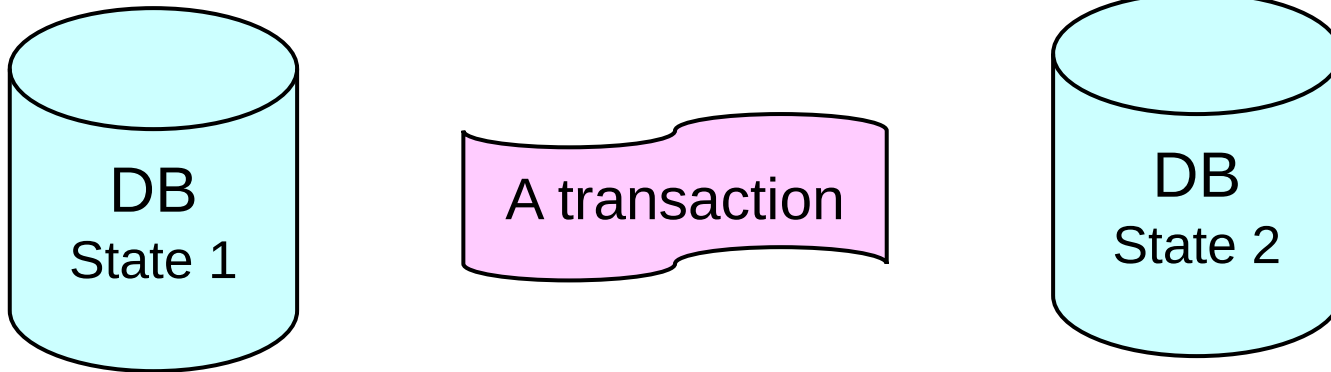
Update Transaction



School of software Yunnan University

■ Consistency

- Complete transactional transformations on data elements bring the database from one consistent state to another.



10.1 Transaction Histories



School of software Yunnan University

- ❑ **Atomic:** The set of updates contained in a transaction must succeed or fail as a unit;
- ❑ **Isolated:** Even though transactions execute concurrently, it appears to each successful transaction that it has executed in a serial schedule with the others.
- ❑ **Durable:** Once a transaction commits, the changes it has made to the data will survive any machine or system failures.
- ❑ **Consistent:** Complete transactional transformations on data elements bring the database from one consistent state to another.

10.1 Transaction Histories



School of software Yunnan University

□ Fundamental Atomic Read and Write Actions

- We use the notation $R_i(A)$ / $R_i(A, val)$ to mean that a transaction T_i performs a read of data item A .
- We also use the notation $W_j(B)$ / $W_j(B, val)$ to mean that a transaction T_j performs a write on data item B .

10.1 Transaction Histories



School of software Yunnan University

□ Transactional Histories with Reads and Writes

- any transaction can be modeled as a series of **reads** ($R_i(A)$) and **writes** ($W_j(B)$), together with **commits** (C_i) arising from the Commit Work statement and **aborts** (A_j) arising because of the **Rollback** Work statement or because of deadlock,
- An "interleaved" series of read and write operations performed by two transactions, **T1** and **T2**, might look like this:

[10.1.2] ... $R_2(A)$ $W_2(A)$ $R_1(A)$ $R_1(B)$ $R_2(B)$ $W_2(B)$ C_1
 C_2 ...

10.1 Transaction Histories



School of software Yunnan University

□ Transactional Histories with Reads and Writes

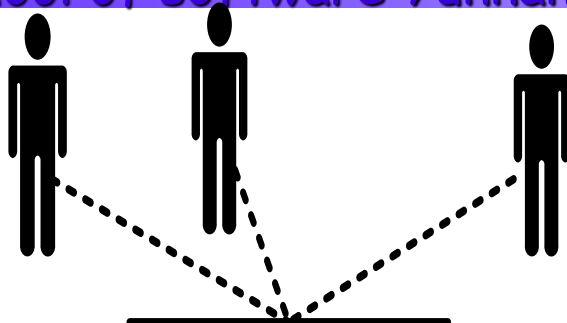
- $A=50$, $B=50$, **T1** is simply adding up the values in the two accounts and printing out, **T2** move 30 units from account A to account B.

[10.1.3] ... $R_2(A, 50)$ $W_2(A, 20)$ $R_1(A, 20)$ $R_1(B, 50)$
 $R_2(B, 50)$ $W_2(B, 80)$ **C1** **C2** ...

10.4 Locking to Ensure Serializability



School of software Yunnan University



Application
programs

Issue calls on behalf of the users:

select , **insert** , **delete** , **update** , **commit** , **rollback**.

Transaction
manager (TM)

Initiates transactions;

Scheduler

Interprets all calls as sequences of **reads** and **writes**;
ensures serializable schedule, using **R** and **W** locks;

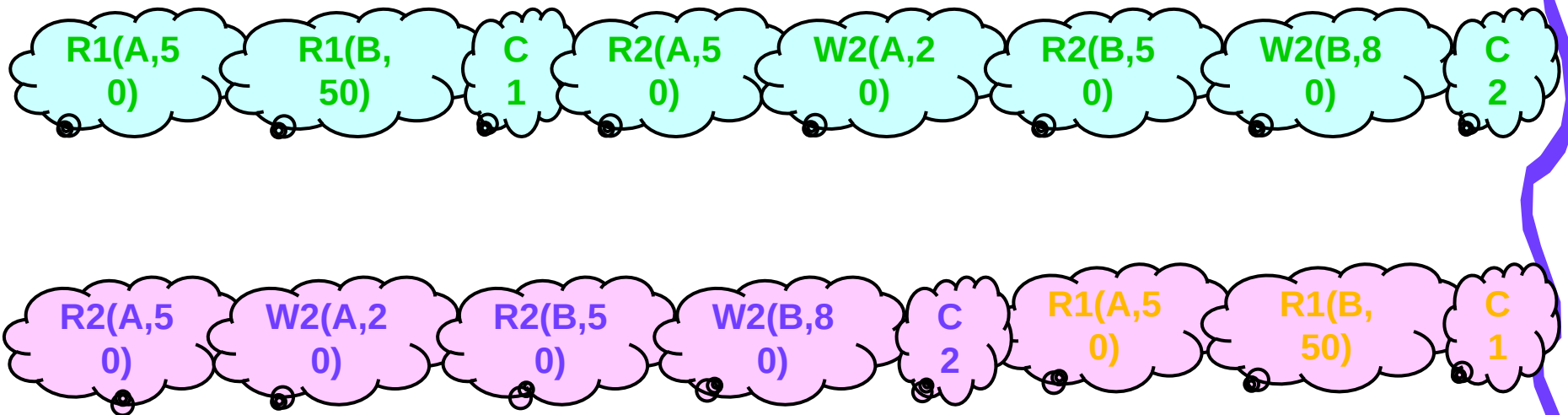
10.4 Locking to Ensure Serializability



School of software Yunnan University

- Two approach:

✂ One simple approach : a strict serial discipline.



10.4 Locking to Ensure Serializability



School of software Yunnan University

∞ Another approach : **locking** that guarantee a serializable schedule with a good deal of interleaving of operations.

□ Two-Phase Locking(2PL)

10.4 Locking to Ensure Serializability



School of software Yunnan University

DEFINITION 10.4.1 Two-Phase Locking(2PL)

- [1] When transaction T_i attempts to read a data item, $R_i(A)$
- the scheduler intercepts this call and first issues a call on its behalf to *read lock* the data item, $RL_i(A)$.
 - Similarly when T_i attempts to write (update) a data item, $W_i(A)$, the scheduler first issues a call on its behalf to *write lock* the data item, $WL_i(A)$.

10.4 Locking to Ensure Serializability



School of software Yunnan University

[2] Before granting a lock on a data item, the scheduler requires the requesting transaction to **WAIT** until **no conflicting lock** on the data item exists.

Two locks on the same data item are said to *conflict* if and only if they are attempted by different transactions and at least one of the two locks is a **write lock**.



10.4 Locking to Ensure Serializability



School of software Yunnan University

[3] There are two phases to locking: the *growing phase*, during which locks are acquired, and the *shrinking phase*, during which locks are released.

The scheduler must ensure that after the shrinking phase begins, no new locks are acquired; it is forbidden for a transaction to release a lock and then acquire another lock at a later time.

10.4 Locking to Ensure Serializability



School of software Yunnan University

■ THEOREM 10.4.2 Locking Theorem.

A history of transactional operations that follows the 2PL discipline is always serializable.

□ R1(A) R2(B) W2(B) R2(A) W2(A) R1(B) C1 C2
R1(50)R2(50)W2(80)R2(50)W2(20)R1(80)C1C2

10.4 Locking to Ensure Serializability



School of software Yunnan University

□ R1(A) R2(B) W2(B) R2(A) W2(A) R1(B) C1 C2

□ RL1(A) R1(A) RU1(A) | RL2(B) R2(B) | WL2(B) W2(B)
WU2(B) | RL2(A) R2(A) | WL2(A) W2(A) | RL1(B)
R1(B) | C1 C2

□ RL1(A) R1(A) | RL2(B) R2(B) | WL2(B) W2(B) | RL2(A)
R2(A) | WL2(A) W2(A) | RL1(B) R1(B) | C1 C2

T2: wait

T1: wait

■ RL2(B) R2(B) | WL2(B) W2(B) | RL2(A) R2(A) |
WL2(A) W2(A) C2 RL3(A) R3(A) RL3(B) R3(B) C3

10.5 Level of Isolation



School of software Yunnan University

- **Theorem 10.4.2 tells us that a scheduler can guarantee serializable histories by imposing a 2PL discipline.**
 - **Property 3 of 2PL assures us that a transaction won't release a lock and later acquire a different lock,**
 - **so we have to hold all locks until there are no remaining new data item accesses to come.**

10.5 Level of Isolation



School of software Yunnan University

- Because of some problem, it was suggested that database system schedulers might weaken the rules of 2PL to reduce the number of transactional access conflicts that cause WAITs.
- This feature is known as *isolation levels*.

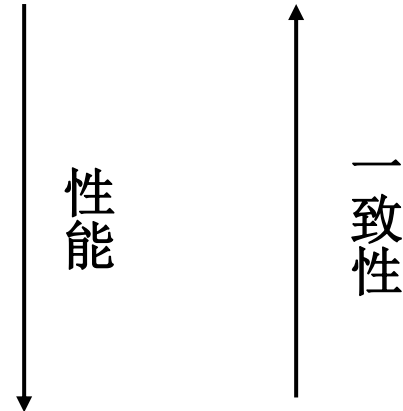
10.5 Level of Isolation



School of software Yunnan University

DB2:

- [1] Repeatable Read (RR)
- [2] Read Stability (RS)
- [3] Cursor Stability (CS)
- [4] Uncommitted Read (UR)



隔离级别



School of software Yunnan University

□ Repeatable Read (RR)

- 锁定应用程序在工作单元中引用的所有行，不能查看其他应用程序的未落实更改；其他应用程序均不能更新、删除或插入可能会影响结果表的行。
- 因此它可以避免各种并发问题。
- 它是DB2的最高级别的隔离级。



□ Read Stability (RS)

- 只锁定应用程序在工作单元中检索的那些行，其他应用程序都不能修改这些行。使用该隔离级的应用程序不能读未提交的数据，可以保护在一个事务中即使多次读取同一行，得到的值也不会改变。
- 但它不能避免新增的幻象行。

隔离级别



School of software Yunnan University

□ Cursor Stability (CS)

- **游标**：使系统为用户开设的一个数据缓冲区，存放 **SQL** 语句的执行结果，用户可以通过游标逐一获取记录。
- 这种隔离级锁定事务工作期间游标所在的行。使用游标稳定性的应用程序不能读未提交数据，假如用户使用这种隔离级，则应用程序读取数据将保持一致性，但不能避免 **不可重复读或幻象读**。
- 对于大多数应用程序需求来说，它通常是首选的。
- 最大并行性。

隔离级别



School of software Yunnan University

□ Uncommitted Read (UR)

- “脏读”，是DB2支持的最低级别的隔离级，它可以允许存取其他应用程序未提交的数据更新。
- 假如用户决定使用这一隔离级，用户的应用程序可能存取不正确的数据。当使用这种隔离级时，可能会出现不可重复读和幻象读现象。
- 只读表，只执行查询的情况下可使用。